ILA — Intelligent Learning Assistant

Complete Build Workflow · Step-by-step guide from zero to a running offline AI tutor

6	~24	8	0
Build phases	Steps total	Core libraries	Internet required

This guide walks you through building ILA from scratch — a fully offline, self-contained AI tutoring system. Follow the phases in order. Each phase produces something tangible you can test before moving on. The system uses Ollama for local LLM inference, ChromaDB for vector storage, LangChain for RAG orchestration, and Streamlit for the UI. No API keys, no cloud calls, no internet dependency once models are downloaded.

How to read this guide

Phase header	Each numbered phase groups related tasks. Do them in sequence — later phases depend on earlier ones being working.
Step card	Each card shows the task name, the library/tool to use (in monospace), and what the step actually does.
Tool stack	At the end, a full recommended tool stack lists every library with its purpose.

1 Environment setup

Get your offline runtime working before any AI code. Estimated time: 1–2 hours.

Install Ollama

```
curl -fsSL https://ollama.com/install.sh | sh
```

Your offline LLM runtime. One-command install on Mac, Linux, and Windows. Handles model downloads, serving, and exposes an OpenAI-compatible API at localhost:11434.

Set up Python environment

```
python -m venv ila_env && pip install ...
```

Python 3.10+ required. Create a virtual environment. Install: langchain, chromadb, pymupdf, sentence-transformers, fastapi, streamlit, plotly.

Pull your LLM

```
ollama pull qwen3:8b
```

Start with qwen3:8b (5 GB) or phi4-mini (3 GB) for low VRAM. Test with: ollama run qwen3:8b — if it responds, you're ready.

Pull your embedding model

```
ollama pull nomic-embed-text
```

Embedding models run separately from the LLM. nomic-embed-text is fast and Apache 2.0. For multilingual content use bge-m3.

2 Document ingestion pipeline

Convert uploaded PDFs/notes into searchable vector chunks — the 'memory' of ILA.

PDF text extraction

```
import fitz # PyMuPDF
```

Parse uploaded PDFs into raw text. PyMuPDF preserves structure, handles tables, headers, and footnotes. For scanned PDFs add pytesseract for OCR.

Chunk the text

```
RecursiveCharacterTextSplitter(chunk_size=512)
```

Split text into ~512-token overlapping chunks (~50 token overlap). LangChain's RecursiveCharacterTextSplitter is the standard starting point.

Generate embeddings

```
ollama.embeddings(model='nomic-embed-text', prompt=chunk)
```

Send each chunk to your embedding model via Ollama's REST API. Returns a vector (list of floats) that encodes the chunk's semantic meaning.

Store in vector database

```
collection.add(documents=chunks, embeddings=vecs, ids=ids)
```

Persist vectors + original text in a local ChromaDB collection. Survives restarts. No internet. Queryable with cosine similarity.

3 RAG query engine

When a student asks a question, retrieve relevant context and feed it to the LLM.

Embed the student's query

```
ollama.embeddings(model='nomic-embed-text', prompt=query)
```

Convert the student's question into a vector using the exact same embedding model used during ingestion. Consistency here is critical.

Retrieve top-k chunks

```
collection.query(query_embeddings=[q_vec], n_results=4)
```

Find the 3–5 most semantically similar text chunks from your vector store. ChromaDB uses cosine similarity by default.

Build the RAG prompt

```
prompt = SYSTEM + context_chunks + student_question
```

Combine: (1) system role instructions, (2) retrieved context passages, (3) the student's question. This is the full input the LLM receives.

LLM generates the answer

```
requests.post('http://localhost:11434/api/generate', json=payload)
```

POST to Ollama's local API. The LLM reads your study material context and produces a grounded answer — no hallucination from thin air.

4 Quiz generation + performance tracking

Auto-generate quizzes from study material and track what the student knows.

Quiz generation via LLM

```
prompt = 'Generate 5 MCQ questions from [context]. Output JSON.'
```

Prompt the LLM to produce structured quiz questions directly from retrieved document chunks. Ask for JSON output: question, options, correct_answer, topic.

Store quiz responses

```
sqlite3 → table: quiz_results(student_id, topic, correct, timestamp)
```

Store every quiz attempt in a local SQLite database. Track: student ID, topic tag, whether correct, timestamp. This feeds the adaptive engine.

Build a knowledge model

```
from pyBKT.models import Model # Bayesian Knowledge Tracing
```

Use Bayesian Knowledge Tracing (BKT) per topic to estimate mastery probability. Simple input: sequence of correct/incorrect responses per topic.

Adaptive topic selection

```
weak_topics = [t for t in topics if mastery[t] < 0.7]
```

Identify topics below mastery threshold. Prioritize these in the next quiz cycle. Pass mastery scores back into the LLM system prompt.

5 Adaptive explanation engine

Personalize how the LLM explains concepts based on student level and history.

Student profile store

```
profile = {topics: {mastery}, style: 'visual', level: 'beginner'}
```

Maintain a per-student JSON profile in SQLite: mastery scores per topic, preferred explanation depth, quiz history, and detected difficulty areas.

Dynamic system prompt injection

```
system = f'Student struggles with {weak}. Use analogies. Level: {level}.'
```

Dynamically build the LLM system prompt using the student's profile. The LLM automatically adjusts explanation style, depth, and vocabulary.

Confusion detection

```
if topic_attempts[topic] >= 2 and score < 0.5: trigger_simpler_explain()
```

If a student asks about the same topic 2+ times or scores below 50%, re-trigger explanation with an explicit 'explain more simply' instruction.

Spaced repetition scheduler

```
next_review = now + interval * ease_factor # SM-2 algorithm
```

Schedule topic reviews using the SM-2 spaced repetition algorithm. Easy topics reviewed less often; difficult ones surface more frequently.

6 Frontend + API layer

Wrap everything in a clean interface students can actually use. Runs entirely in browser.

FastAPI backend

```
uvicorn main:app --host 0.0.0.0 --port 8000
```

Expose clean REST endpoints: POST /upload (ingest PDF), POST /ask (RAG query), GET /quiz (generate questions), GET /progress (mastery data). Everything calls Ollama locally.

Streamlit chat UI

```
import streamlit as st → st.chat_input() + st.chat_message()
```

Chat interface with message history, source citations from retrieved chunks, and a sidebar showing current topic and mastery level.

PDF upload handler

```
uploaded = st.file_uploader('Upload study material', type='pdf')
```

Drag-and-drop PDF uploader → triggers ingestion pipeline (phases 2) → shows progress bar → confirms 'X chunks indexed' when complete.

Progress dashboard

```
import plotly.express as px → px.bar(mastery_df, x='topic', y='score')
```

Visual dashboard: mastery bar chart per topic, quiz streak counter, time-spent heatmap, weak areas highlight. All data from local SQLite.

Recommended tool stack

Ollama Local LLM + embedding runtime. OpenAI-compatible API at localhost:11434.	**ChromaDB** Local vector database. Persistent, no server needed, Python-native.
LangChain RAG orchestration — chains, prompts, text splitters, retrieval.	**PyMuPDF (fitz)** Fast PDF text extraction. Handles tables, layout, and metadata.
FastAPI Backend REST API. Async, fast, auto-generates docs at /docs.	**Streamlit** Frontend UI. Chat interface, file uploader, charts in ~100 lines.

SQLite (built-in) Student performance store. Zero config, file-based, ships with Python.	**pyBKT** Bayesian Knowledge Tracing for per-topic mastery estimation.
sentence-transformers Alternative embedding pipeline (HuggingFace models directly).	**Plotly / Altair** Progress dashboard charts — mastery bars, heatmaps, streak counters.
pytesseract OCR for scanned PDFs. Wraps Tesseract engine.	**Unstructured** Advanced document parsing — handles Word, HTML, images, tables.

Quick-start checklist

Phase 1	[ ] Install Ollama → pull qwen3:8b → confirm ollama run qwen3:8b responds
Phase 1	[ ] Create Python venv → pip install langchain chromadb pymupdf streamlit fastapi
Phase 2	[ ] Write ingest.py: PDF → chunks → embeddings → ChromaDB. Test on one PDF.
Phase 3	[ ] Write query.py: user question → embed → retrieve chunks → LLM answer. Test in terminal.
Phase 4	[ ] Add quiz_gen.py: prompt LLM for JSON quiz from retrieved chunks. Store in SQLite.
Phase 5	[ ] Add student_profile.py: load/save mastery scores. Inject into system prompt.
Phase 6	[ ] Build app.py in Streamlit: chat UI + PDF uploader + /progress page.
Phase 6	[ ] Add FastAPI backend wrapper (optional, useful if building a multi-user version).

System architecture overview

Component	Role in ILA	Technology
Document store	Indexed study material chunks	`ChromaDB + PyMuPDF`
Embedding model	Semantic vectorisation of text	`nomic-embed-text / bge-m3`
LLM	Explanation, quiz, Q&A; generation	`qwen3:8b / deepseek-r1:14b`
RAG engine	Query → retrieve → prompt → answer	`LangChain + Ollama API`
Knowledge model	Per-topic mastery tracking	`pyBKT / SQLite`
Adaptive engine	Personalised prompt injection	`Python + SQLite profiles`
Backend API	HTTP endpoints for the frontend	`FastAPI`
Frontend UI	Chat, upload, quiz, dashboard	`Streamlit`